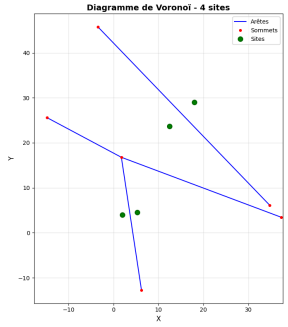
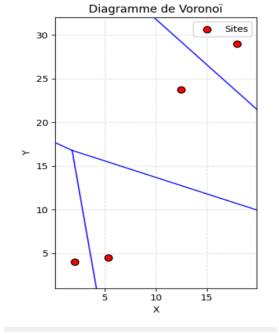
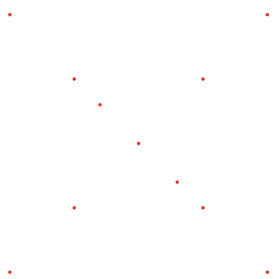
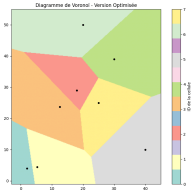
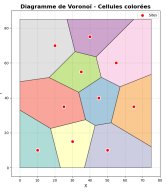
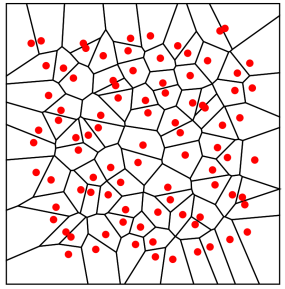
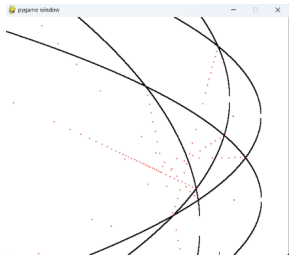
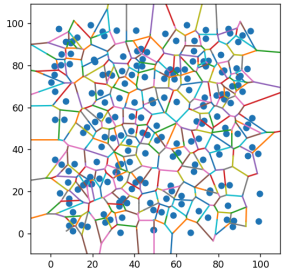
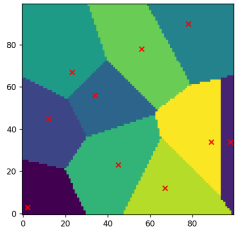
Synthèse et comparaison des modèles IA

Afin de comparer les différents modèles d'IA générative, nous avons choisi de tous avoir le même prompt de départ construit à partir des consignes (Cf. annexe 1). De plus, nous avons synthétisé nos différentes solutions dans un tableau comparatif que voici :

Nom du LLM	Le résultat répond au cahier des charges ?	Temps passé à faire des itérations	Problèmes rencontrés	Résultat du diagramme de voronoï	Mesure de la performance
Claude sonnet 4.5	Pratiquement, excepté le calcul du nombre de sommet et l'aspect "joli"	~1h	Mauvaise implémentation de l'algorithme de fortune, et prolongement des segments ouvert en fin d'algorithme		$O(n^3)$ et 11,2 secondes en moyenne pour 200 points
Grok 4.1	Oui sauf ajout d'une fonctionnalité pour mesurer les performances en fonction du nombre de points chargés sur l'interface tkinter	47 mins 31	- Tests qui ne passent pas -Affichage mauvais en raison d'une échelle du diagramme trop grande		$O(n^3)$ 100 points : ~0,9s 200 points : ~4,5s 500 points : ~57s

Mistral IA Le Chat	Cahier des charges très partiellement rempli (uniquement la représentation des points)	50 min 45 secondes	- Dysfonctionnement des chemins d'accès aux modules proposés par le modèle. - Algorithme implémenté dans un script mais non fonctionnel lors de l'appel. - Malgré de nombreuses tentatives, code qui détecte bien les points mais ne réalise pas de diagramme.		
Gemini	Pratiquement bon avec l'algorithme de balayage point par point	~ 1h15	problème d'environnement et de version avec python, d'optimisation des calculs, conflit avec l'interface		O(W*H*N) où W et H la hauteur et largeur de la grille 2.42 secondes pour 100 points et 23 secondes pour 1000 points
Deepseek	Cahier des charges remplis	~2h	- Assez long pour envoyer le rendu de chaque fichier (il a pris un moins de 10 minutes pour m'écrire tous les fichiers de base) - Également, il a créé		$O(n^3)$ 100 points: 0.02527s 200 points: 0.10570s 500 points: 0.76888s

			des fichiers / dossiers qui restent vides et inutilisés jusqu'à la fin - Il fait pleins de petites erreurs et d'oublies, comme utilisé une bibliothèque dans le code mais il oublie de l'importer ou bien il confond x et y dans une fonction		
ChatGPT	Le résultat est rempli avec un algorithme naïf simple, bien que l'esthétique soit rudimentaire	~ 10 minutes	Résultat produit instantanément, un fichier de configuration manquant qu'il a fallu demander puis redemander corrigé. Les points n'étaient pas affichés au départ seulement les traits et l'affichage était trop petit. Il a fallu demander de corriger à ce niveau là également.		$O(n^3)$ 4 points : 0.000158 secondes 100 points : 0.083414 secondes 500 points : 6.319317 secondes
Perplexity					

Développement humain - Ioan	Résultat satisfaisant pour un premier jet avec une sweepline et le calcul de toutes les intersections entre les paraboles grâce à une modélisation géométrique complète	~ 10h	Pas de beachline donc détection d'intersections inutiles et diagramme final incomplet, c'est l'étape 1 de la réalisation de l'algorithme de fortune		
Développement humain - Lucas	si ce n'est la batterie de tests unitaire et le prolongement des segments le résultat est satisfaisant	15~20h	Algorithme simple à comprendre mais difficile à implémenter.		O(n*racine n*log n) car j'ai oublié une petite optimisation dans ma recherche de cercle, au lieu d'en tester 3 j'en teste pour la longueur de la plage. 200 points en 2s.
Développement humain - Razi, Lorenzo, Vincent, Léona, Salma	En terme d'optimisation et respect des bonnes pratiques du code, c'est faible, mais le résultat et l'affichage sont bons	6h	Problème avec matplotlib pour l'affichage, on a donc utilisé imshow() pour simplifier le coloriage		O(h*l*g) où h et l la hauteur et largeur du graphique, et g le nombre de germe

Réflexion

Quand on compare les temps pour obtenir un résultat entre un développement IA et humain, la concurrence est déloyale et l'IA l'emporte haut la main, mais ce résultat n'est il pas biaisé ? En ayant commencé par une phase de développement sans IA, avec de la recherche approfondie sur le sujet, notre expertise est devenue suffisante pour être efficace dans la phase de vibe coding. Cependant en ayant fait le parcours inverse il paraît vraisemblable d'imaginer que le développement avec IA aurait été plus long.
De plus, le sujet de voronoï pour des diagrammes sans contraintes et avec des distances euclidiennes est bien documenté et de nombreux projets github existent sur des implémentations efficaces. C'est pourquoi l'IA s'en tire si bien. Cependant, la majorité des projets auxquels nous serons confrontés dans notre carrière de développeurs seront autrement plus complexes et pour peu qu'on nous demande une innovation, la maîtrise de sujet en profondeur sera indispensable. Savoir identifier les forces et les faiblesses de l'IA et en faire une utilisation raisonnée en phase d'apprentissage est la clé d'une montée en compétence régulière. La veille technologique sur ce sujet est également à l'ordre des priorités pour nous en tant que développeur junior ou étudiants en bac+3 et au-delà. Les notions d'agent ou de MCP sont intéressantes à connaître suffisamment pour contourner les pièges de l'utilisation abusive de l'IA et en tirer des bénéfices.

Annexes

Annexe 1 : Prompt d’origine pour les différentes solutions de la phase 2

Rôle:
Tu es un ingénieur en informatique et tu es très attaché aux principes du Clean Code.

Consigne :
Diagrammes de Voronoï → Votre équipe a été sollicitée pour proposer une application qui permet de déterminer un diagramme de Voronoï à partir d'une liste de points du plan, et permettre sa visualisation. Phase 1 Vous devrez vous acquitter de cette tâche en proposant une interface conviviale à partir de fichiers qui contiennent une liste de points sous forme de paire de nombres (les coordonnées) au rythme de une par ligne : 2,4 5.3,4.5 18,29 12.5,23.7 L'application permettra de visualiser le diagramme obtenu et éventuellement exporter le résultat sous forme de fichiers SVG ou image.
L'application fournie devra respecter le plus haut niveau de bonnes pratiques de programmation. Elle incluera donc aussi notamment une série de tests.

Spécifications :
- Tu ne dois pas utiliser la librairie intégrée à python qui trace directement le diagramme de voronoï mais bien recoder un algorithme complet avec seulement la librairie math de python.
- Fais une arborescence cohérente en respectant les principes SOLID.
- Inclus également un moyen de mesurer les performances de l’algorithme en fonction du nombre de points dans le fichier txt.
- Le code doit inclure des docstrings clairs pour chaque classe et fonction, qui doivent par ailleurs avoir un nom explicite, ainsi que des commentaires pertinents.
- Le projet doit contenir un README avec les instructions pour créer le venv python et installer les requirements etc…

Tests :
Tu dois impérativement fournir une suite de tests unitaires pytest pour valider la lecture du fichier, la gestion des erreurs et la logique de calcul, en respectant les principes du TDD qui font partie du Clean Code.

Livrable attendu de ta part :
Génère l'intégralité du code source structuré (tu dois diviser en plusieurs fichiers en indiquant les noms de fichiers et les imports).

Annexe 2 : workflow de la solution Claude Sonnet 4.5

Prompt 1 : verrifie les versions dans mon requirements.txt car j'ai une erreur

Erreur corigée avec les bonnes versions compatibles entre elles

Prompt 2 : bon malheureusement , le résultat du calcul pour mes 4 points n'est pas tout à fait corrects , certaints segments sont faux et le diagramme final n'est pas le même que celui qui serait généré si jutilisait la librairie dédié de python donc il y a une erreur quelque part, le reste du fonctionnement de l'application est très bien par contre donc concentre toi simplement sur l'erreur dans l'algorithme!

Erreur corigée dans le calcul des perpendiculaires (29 lignes changées)

Prompt 3 : très bien , le diagramme est maintenant correct, cependant il manque une dernière étape, à la fin il faudrait que tous les segments encore ouvert soient refermée pour couper le cadre et ne pas finir avec des segments qui s'arrètent sans toucher le bord du diagramme donc arrange toi pour prolonger les segments correctements

Ecriture d'une methode _close_cells(self) mais qui ne fait que relier les segments ouvert en fin d'algorithme avec d'autres sommets ... ajout de 200 lignes de code !

Prompt 4 : les segments ne doivent pas se refermer avec d'aurtes sommets ...

pour prolonger les segments restant en fin d'algorithme il faudrait avancer artificiellement la ligne de balayage plus loin pour calculer dans quels direction doivent se prolonger les segmentss qui sont sencé partir des sommets (il ne peux pas y avoir de sommets avec que 2 segments qui en parte...) donc ajuste cette fonction close_cells et change le nom par , last_step_to_prolonge_segments ou quelquechose dans ce style.

Il n'a pas réussi (en enlevant 200 lignes et en ajoutant 150 lignes)

Prompt 5 : bon ça n'a pas marché ... on va faire plus simple, en fin d'algo si un sommet n'a qu'un segment relié alors il doit y avoir un segment qui part de ce sommet dans la même direction que ce segment jus'quà atteinre le bord de la boite. Si un sommet n'a que 2 segments qui lui sont relié, le segment que tu doit créer pour rejoindre le bord de la boite doit avoir comme direction la bissectrice de langle entre les deux segments. ainsi tu prolonge les segments qui arrivent à un sommet tout seul et tu créer un segment pour les sommet avec 2 segments

Ça ne marche pas non plus , je vais me contenter de cela...

Prompt 6 : en fait j'ai compris, tu agrandis la zone pour quelle soit plus grande que les points qu'elle contient donc c'est sans fin , fait un cadre en pointillés qui ne grandit pas

Erreur corigées mais mauvais nombre de sommets, il y a encore des choss à améliorer surtout que la complexité semble être en O(n^3) selon les mesures.

Annexe 3 : workflow de la solution Grok 4.1

Prompt 1 : “Il manque quelques informations et des tests que tu n'a pas écrit : où doit-on trouver exactement le fichier texte points.txt pour les germes.
Les tests manquants : test_file_handler.py, test_performance.py
De plus, toujours dans les tests, tu as oublié certains imports.”

Résultat : Grok m’explique qu’avec l’interface tkinter il suffit juste d’importer le fichier .txt et donc il peut se placer là où on le souhaite mais il est quand même préférable de le garder à la racine du dossier. Pour les tests, ils ont été bien générés cette fois-ci mais un test ne passe pas, celui qui doit parser et bien récupérer les coordonnées des germes depuis un fichier texte avec des commentaires et des lignes vides.

Prompt 2 : “Le résultat obtenu du diagramme n'est pas bon. Il y a aussi un test qui ne passe pas je t'envois l'erreur : ” + envoi de l’erreur de pytest + Capture d'écran du diagramme.

Résultat : Le diagramme n'a pas affiché un bon résultat car l’échelle du diagramme était trop grande pour des points qui se trouvait assez proche et donc les points et les arêtes n’étaient pas conformes à ce que j’attendais. L’erreur du test persiste malgré la modification d’un fichier.

Prompt 3 : “L'affichage n'est toujours pas bon. En fait les germes s'affichent (les 4 points) cependant l'échelle du diagramme est trop grande. Il faut que le diagramme puisse s'adapter en fonction des coordonnées des points.
De plus, des tests ne passent pas, voici l'erreur : ” + envoi de l’erreur de pytest (ValueError sur float) + Capture d'écran du diagramme.

Résultat : Le problème du test précédent a été réglé en modifiant certains fichiers mais un autre est apparu. Je décide de revenir à une version antérieure du code et de ne modifier que le fichier plotter.py qui permet maintenant de réguler les dimensions du diagramme (du plan) en fonction des coordonnées des germes. L’affichage est corrigé et correspond bien à un diagramme de Voronoï.

Prompt 4 : “ok pour l'affichage il suffisait juste de changer la visualisation dans plotter.py. Le test suivant ça ne passe toujours pas : ” (celui avec commentaires et espaces, le même que dans les derniers prompts) + envoi de l’erreur de pytest

Résultat : Plusieurs fichiers ont été modifiés mais ça ne réglait toujours pas le problème (au lieu de 9 tests sur 10 validés il y en avait que 8 à présent qui sont passés) alors j’ai décidé de lui envoyer le fichier qui avait des tests qui ne passait pas et d’effacer les dernières modifications.

Prompt 5 : “donne moi tout le code du file_handler.py”
Résultat : En lui envoyant le fichier complet qui avait un test qui ne passait pas, les tests sont tous passés.

Annexe 4 : workflow de la solution DeepSeek

Prompt 1: “en essayant de passer les tests, j'ai eu 2 erreurs sur 37, peux tu me les corriger en se basant sur la sortie des tests. Renvoie moi tous les codes que tu auras corrigé pour que je puisse faire un copier coller directement sur visual code studio et vérifier que tous les tests passent” + erreur des tests

Résultat : Il a essayé de me réécrire certaines parties du code mais, le résultat des tests étaient toujours le même. Il n’a pas compris ce qu’il devait modifier ni les erreurs qu’il avait faites.

Prompt 2: “c’est toujours faux, modifie le test car il me semble que le code est bon mais c'est le test il fait quelque chose de bizarre, alors essaye de modifier le test pour le faire passer au vert et renvoie moi le fichier de tests au complet” + erreur des tests

Résultat : Il a réussi à me débloquer certains passages et certaines erreurs, mais j’ai dû vérifier par moi même pour rectifier les petits détails qui manquaient. Il a également ajouté de nouveaux tests sans que j’ai eu besoin de le lui demander. Lorsque que je l’ai testé il a réussi à afficher un résultat mais il y avait des incohérences et il n’était pas très “jolie” à voir.

Prompt 3: “le résultat que tu m'as fait ne correspond pas au résultat attendu, j'aimerais que le résultat soit avec des couleurs et qu'il n'y ai pas de droite parasite, fait en sorte que le résultat fonctionne et corresponde bien au résultat d'un diagramme de voronoï” + image du résultat que j’attendais

Résultat : Il m’a modifié du code et m’a fait ajouter de nouveaux fichiers, pour au final obtenir le résultat que j’attendais. Il a pris du temps à comprendre ces erreurs et a fait beaucoup de petits oublis que j’ai modifiés par moi-même.

Annexe 5 : workflow de la solution Gemini

Prompt 1: “les versions python, matplotlib et pytest ne sont pas compatibles avec ton programme”

Résultat : Il a optimisé le programme pour python 3.14 et a crée un fichier requirements.txt avec les bonnes versions

Prompt 2: “peux tu me restructurer ton projet avec une séparation nette entre les données et le engine”

Résultat : Mise en place des fichiers models, engine et io_handler (lecture de fichiers). Cette architecture a permis de rendre le code clair et respectant l’architecture SOLID.

Prompt 3: “il y’a un problème d’affichage dans le graphique, il faut que tu adapte l'échelle des points en fonction de la résolution du graphique, parce que aller jusqu’à 400 alors que les points sont autour de 20 rend l’affichage mal fait ”

Résultat : Il a ajouté une méthode pour trouver les coordonnées min/max et rendu l’affichage dynamique pour faire correspondre les pixels aux valeurs réelles des points

Prompt 4: “fournis moi d’autres tests qui englobent une grande partie du programme, et gèrent les cas d’erreurs”

Résultat : Il m’a généré des tests complets et gérant les cas d’erreurs

Prompt 5: “trouve moi une autre solution pour améliorer la vitesse de calcul”

Résultat : Passage à la distance au carré et suppression du sqrt coûteux. Mise en cache des coordonnées dans des tuples pour accélérer l'accès mémoire.

Annexe 6 : workflow de la solution Mistral IA Le Chat

Prompt 1 : voir annexe n°1

Durée de traitement de la requête par Mistral : 20 secondes
Durée de traitement par un humain pour l'adapter et l'exécuter : 8 minutes

Prompt 2 : Implémente l'algorithme de voronoi dans voronoi.py, implémente l'exportation en svg dans svg_visualizer.py et ajoute une série de points de tests dans example_points.txt.

Durée de traitement de la requête par Mistral : 30 secondes
Durée de traitement par un humain pour l'adapter et l'exécuter : 5 minutes

Prompt 3 : Lors de l'exécution de main.py, j'ai cette erreur :
Traceback (most recent call last):
File "main.py", line 1, in <module>
from parsers.file_parser import FileParser
File "src/parsers/file_parser.py", line 2, in <module>
from ..models.point import Point
ImportError: attempted relative import beyond top-level package
Corrige cette erreur stp

Durée de traitement de la requête par Mistral : 5 secondes
Durée de traitement par un humain pour l'adapter et l'exécuter : 2 minutes

Prompt 4 : Lors de l'exécution de main.py, j'ai cette erreur :
Traceback (most recent call last):
  File "src/main.py", line 1, in <module>
    from parsers.file_parser import FileParser
  File "src/parsers/file_parser.py", line 2, in <module>
    from src.models.point import Point
ModuleNotFoundError: No module named 'src'
Corrige cette erreur stp

Durée de traitement de la requête par Mistral : 10 secondes
Durée de traitement par un humain pour l'adapter et l'exécuter : 2 minutes

Prompt 5 : J’ai fait la commande « python -m src.main.py » dans la racine du projet (voronoi/) et j’ai cette erreur :
Traceback (most recent call last):
  File "/anaconda3/lib/python3.9/runpy.py", line 188, in _run_module_as_main
    mod_name, mod_spec, code = _get_module_details(mod_name, _Error)
  File « /anaconda3/lib/python3.9/runpy.py", line 111, in _get_module_details
    __import__(pkg_name)
  File "voronoi/src/main.py", line 1, in <module>
    from parsers.file_parser import FileParser
ModuleNotFoundError: No module named 'parsers’
Corrige cette erreur stp.

Durée de traitement de la requête par Mistral : 10 secondes
Durée de traitement par un humain pour l’adapter et l’exécuter : 5 minutes

Prompt 6 : J'ai bien des __init__.py (fichiers vides) dans tout les dossiers, les chemins sont des chemins absolus pour importer les modules et j'ai cette erreur quand j'exécute "python -m src.main" depuis voronoi/.
Pourquoi j'ai encore cette erreur ?
Traceback (most recent call last):
File "anaconda3/lib/python3.9/runpy.py", line 197, in _run_module_as_main
return _run_code(code, main_globals, None,
File "anaconda3/lib/python3.9/runpy.py", line 87, in _run_code
exec(code, run_globals)
File "voronoi/src/main.py", line 1, in <module>
from parsers.file_parser import FileParser
ModuleNotFoundError: No module named 'parsers'

Durée de traitement de la requête par Mistral : 10 secondes
Durée de traitement par un humain (réflexion sur le souci qu’il n’est pas capable de résoudre) : 10 minutes

Refacto personnelle après avoir repéré le souci : 5 minutes

Prompt 7 : Le svg produit par le code ne comporte que des points, on ne voit pas les zones du diagramme de voronoi associé. Remède y stp.

Durée de traitement de la requête par Mistral : 20 secondes
Durée de traitement par un humain pour l’adapter et l’exécuter : 3 minutes

Prompt 8 : Le code fonctionne lorsque j’exécute main.py mais il renvoie :
Temps d'exécution: 0.0001 secondes
Aucune arête ou point à afficher. Or j’ai bien des points dans example_points.txt. De ce fait pourquoi il ne réalise pas le diagramme de voronoï ?

Durée de traitement de la requête par Mistral : 15 secondes
*Durée de traitement par un humain pour l'adapter et l'exécuter :*3 minutes

Prompt 9 : Il ne détecte toujours aucun point

Durée de traitement de la requête par Mistral : 5 secondes
Durée de traitement par un humain pour l'adapter et l'exécuter : 2 minutes

Prompt 10 : Le code renvoie :
Chemin du fichier:
/Users/lorenzojaccaudgodefroy/Documents/ÉTUDES/ÉTUDES_SUP_IUT_VILLETANEUSE_LORENZO/BUT3_Info/SAÉ_Voronoï/Voronoï mistral/voronoi/src/example_points.txt
Ligne lue: '19,23'
Point ajouté: (19.0, 23.0)
…
Point ajouté: (76.0, 80.0)
Nombre de points lus: 10
Points après parsing: [Point(19.0, 23.0), Point(12.0, 42.0), Point(35.0, 15.0), Point(28.0, 30.0), Point(5.0, 10.0), Point(40.0, 25.0), Point(22.0, 5.0), Point(30.0, 40.0), Point(10.0, 20.0), Point(76.0, 80.0)]
Nombre de points en entrée: 10
Temps d'exécution: 0.0001 secondes
Nombre de points en entrée: 10
Aucune arête ou point à afficher.
Fait en sorte qu'il construise le diagramme de voronoi.

Durée de traitement de la requête par Mistral : 10 secondes
Durée de traitement par un humain pour l'adapter et l'exécuter : 4 minutes

Prompt 11 : Implémente un autre algorithme pour générer le diagramme

Durée de traitement de la requête par Mistral : 10 secondes
Durée de traitement par un humain pour l'adapter et l'exécuter : 2 minutes

Prompt 12 : Modifie le fichier svg_visualizer pour qu'il fonctionne avec cet algo

Durée de traitement de la requête par Mistral : 20 secondes
Durée de traitement par un humain pour l'adapter et l'exécuter : 2 minutes

Annexe 7 : workflow de la solution ChatGPT

Prompt 1 :
Rôle: Tu es un ingénieur en informatique et tu es très attaché aux principes du Clean Code. Consigne : Diagrammes de Voronoï → Votre équipe a été sollicitée pour proposer une application qui permet de déterminer un diagramme de Voronoï à partir d'une liste de points du plan, et permettre sa visualisation. Phase 1 Vous devrez vous acquitter de cette tâche en proposant une interface conviviale à partir de fichiers qui contiennent une liste de points sous forme de paire de nombres (les coordonnées) au rythme de une par ligne : 2,4 5.3,4.5 18,29 12.5,23.7 L'application permettra de visualiser le diagramme obtenu et éventuellement exporter le résultat sous forme de fichiers SVG ou image. L'application fournie devra respecter le plus haut niveau de bonnes pratiques de programmation. Elle incluera donc aussi notamment une série de tests. Spécifications : - Tu ne dois pas utiliser la librairie intégrée à python qui trace directement le diagramme de voronoï mais bien recoder un algorithme complet avec seulement la librairie math de python. - Fais une arborescence cohérente en respectant les principes SOLID. - Inclus également un moyen de mesurer les performances de l'algorithme en fonction du nombre de points dans le fichier txt. - Le code doit inclure des docstrings clairs pour chaque classe et fonction, qui doivent par ailleurs avoir un nom explicite, ainsi que des commentaires pertinents. - Le projet doit contenir un README avec les instructions pour créer le venv python et installer les requirements etc… Tests : Tu dois impérativement fournir une suite de tests unitaires pytest pour valider la lecture du fichier, la gestion des erreurs et la logique de calcul, en respectant les principes du TDD qui font partie du Clean Code. Livrable attendu de ta part : Génère l'intégralité du code source structuré (tu dois diviser en plusieurs fichiers en indiquant les noms de fichiers et les imports).

Résultat : M'a fourni le schéma d'architecture propre avec séparation des responsabilités. J'ai le contenu de mes fichiers source, mes fichiers d'infrastructure pour l'exportation svg, trois fichiers de tests ainsi que toutes les informations d'exécution et d'installation dans un environnement virtuel venv, un readme ainsi qu'un fichier requirements.
Cette étape m'a pris environ 5 min pour constituer mon architecture et copier tous mes fichiers.

Prompt 2 :
à quoi sert pyproject.toml? il est vide?

<u>Résultat:</u> A fourni le fichier pyproject.toml qui était présent dans le premier prompt mais uniquement dans l'architecture mais dont le contenu était manquant.
Cette étape m'a pris 1 min pour corriger.

Prompt 3 :

```
(venv) PS C:\Users\User\Documents\Programmation\BUT3\Voronoi> python -m
 src.voronoi.main input.txt output.svg
Traceback (most recent call last):
  File "C:\Users\User\AppData\Local\Programs\Python\Python39\lib\runpy.py", line
197, in _run_module_as_main
    return _run_code(code, main_globals, None,
  File "C:\Users\User\AppData\Local\Programs\Python\Python39\lib\runpy.py", line
87, in _run_code
    exec(code, run_globals)
  File
"C:\Users\User\Documents\Programmation\BUT3\Voronoi\src\voronoi\main.py", line
2, in <module>
    from voronoi.application.file_reader import PointFileReader
ModuleNotFoundError: No module named 'voronoi'
```

Résultat: A corrigé le fichier pyproject fourni précédemment et donné des instructions d'activation pour la détection des modules.
Cette étape m'a pris 1 min pour corriger.

Prompt 4 :
fournis moi le input.txt, dis moi où le mettre (respectes les coordonnées de la consigne)

Résultat: A fourni le fichier input.txt et m'a indiqué où le mettre dans le projet.
Cette étape m'a pris 2 min pour implémenter et vérifier les résultats.

Prompt 5 :
je ne vois pas mes points et actuellement le résultat est tout petit

Résultat: A corrigé le fichier d'export svg et le fichier pyproject de configuration pour respecter ma demande.
Cette étape m'a pris 2 min pour corriger mes fichiers et vérifier les résultats.

Toutes les réponses aux prompts ont été exécutées instantanément et n'ont pris que quelques secondes à charger complètement.